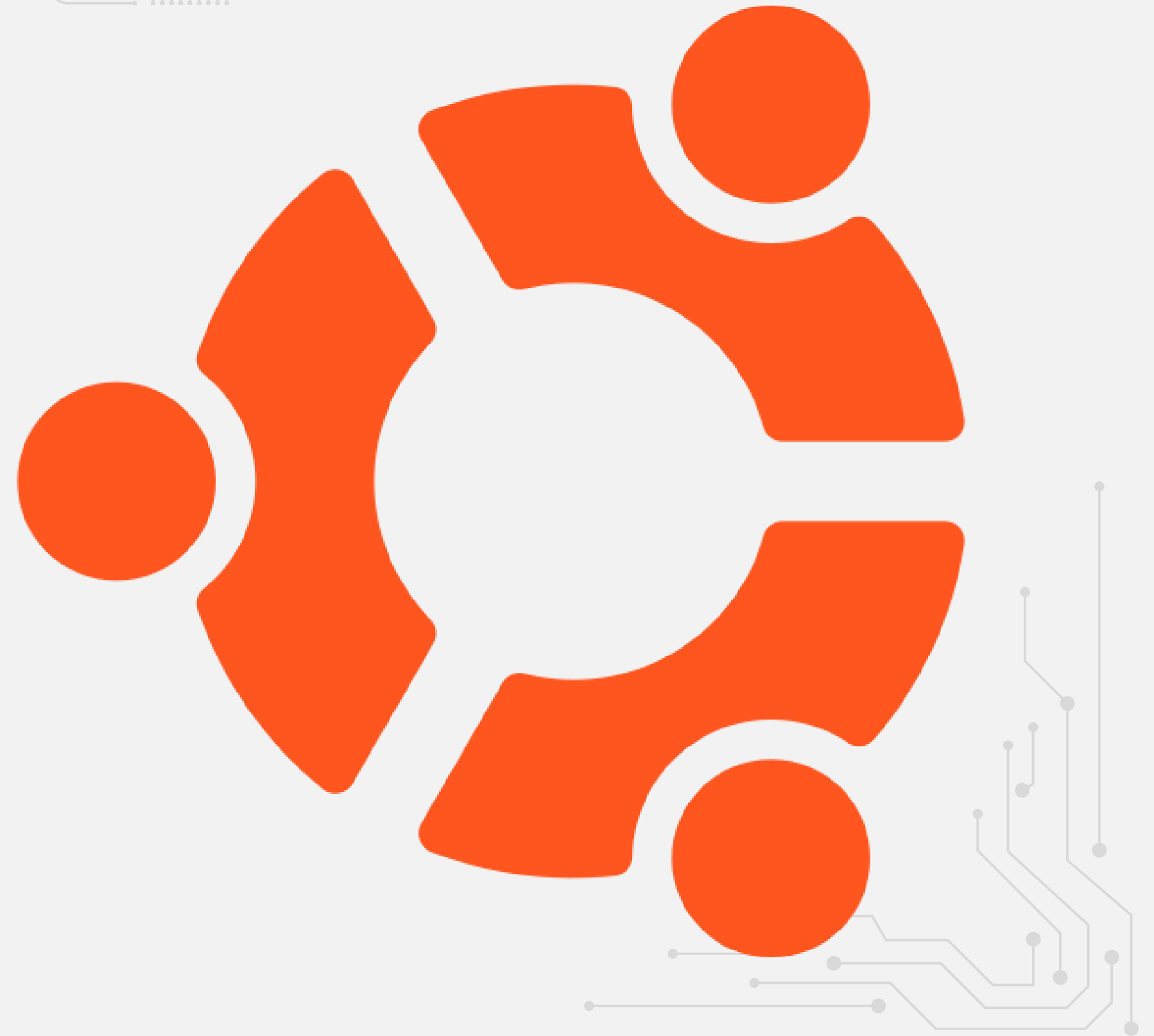


 Scarlet Abreu - 10153953

Proyecto Integrador Final: Diseño, Aseguramiento y Observabilidad de un Servicio Linux en Producción

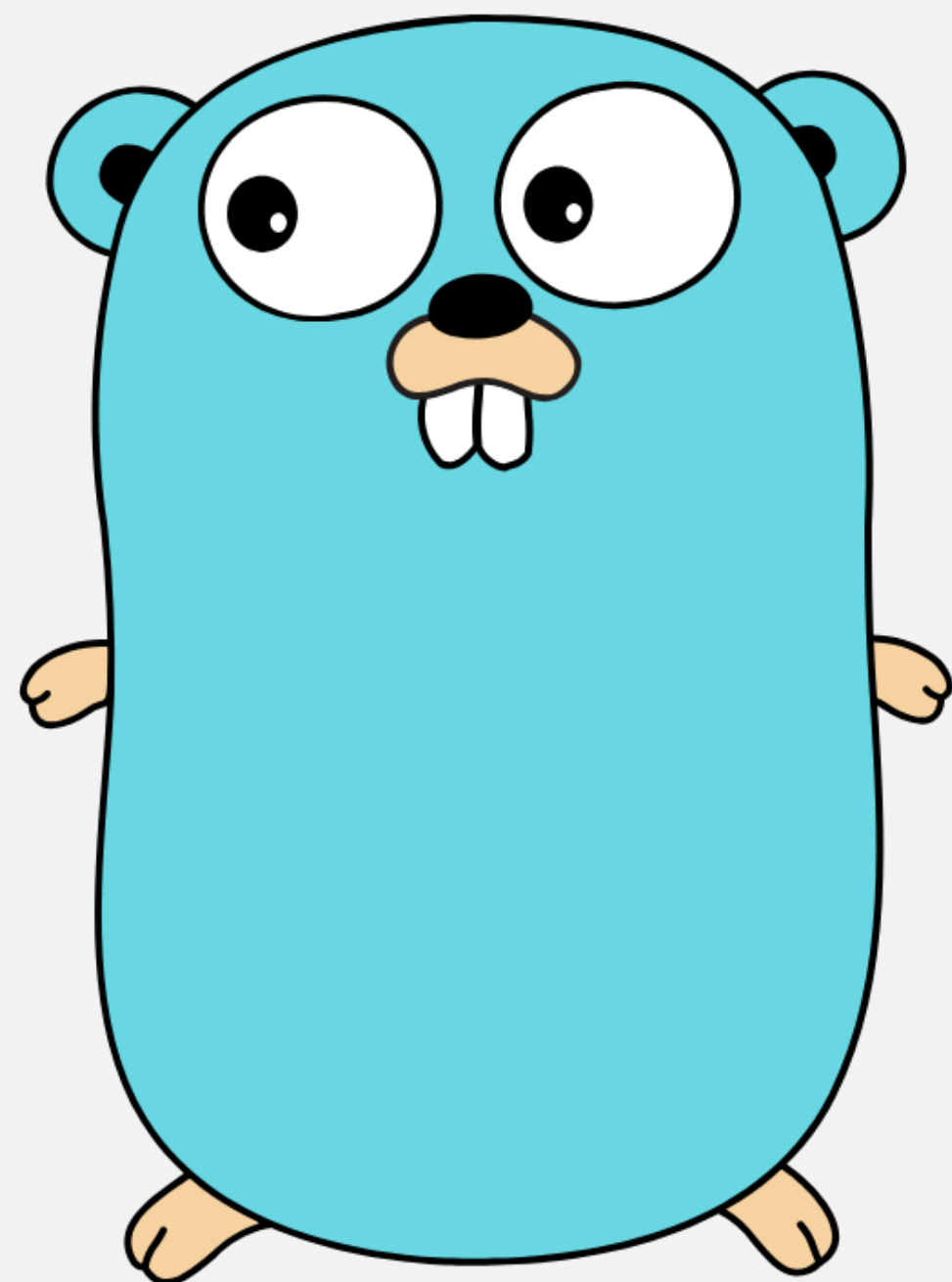
Sistemas Operativos - ICC331



Integración de Tecnologías DevOps, Seguridad y Administración de Sistemas Operativos

- **Objetivo Principal:** Desarrollar, desplegar y asegurar un servicio backend basado en Go (Golang) en un entorno de servidor Linux (Ubuntu).
- **Alcance:** Demostrar la implementación de prácticas de ingeniería de software, incluyendo:
 1. Gestión de la Robustez mediante systemd.
 2. Aseguramiento de Acceso Remoto vía SSH con Clave Pública y UFW (Firewall).
 3. Desarrollo de un binario compilado en Go para alto rendimiento.
 4. Distribución y Portabilidad mediante Empaquetado Binario (.deb).





Arquitectura del Despliegue

- **Entorno Anfitrión (Host):** PC Windows (Scarlet Abreu) utilizada para la administración remota (SSH) y las pruebas de conectividad (curl, Test-NetConnection).
- **Máquina Virtual (VM):** Ubuntu Linux (LTS) donde reside el servicio.
- **Conexión Asegurada:** El acceso a la VM se realiza exclusivamente a través de SSH mediante autenticación de clave pública.
- **Servicio Principal:** goservice (Go Binario) gestionado por systemd y ejecutándose en el puerto 9090/tcp.
- **Seguridad Perimetral:** El Firewall UFW permite únicamente el tráfico entrante a los puertos 22 y 9090.



Aseguramiento del entorno

```
z:\Users\Scarlet>ssh -p 2222 scarlet@127.0.0.1
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Dec 15 10:02:43 PM UTC 2025

System load:          0.15
Usage of /:            13.2% of 24.44GB
Memory usage:         11%
Swap usage:           0%
Processes:            126
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:feda:5afc

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun Dec 14 23:45:35 2025 from 10.0.2.2
scarlet@LTS:~$
```

Autenticación

El acceso es solo con clave pública,
deshabilitando la autenticación menos segura
por contraseña.

Permisos Estrictos

Permisos 600 y 700 para el directorio ~/.ssh. Nadie más que
el propietario puede leer las claves.

```
scarlet@LTS:~$ ls -ld ~/.ssh
drwx----- 2 scarlet scarlet 4096 Dec 14 20:07 /home/scarlet/.ssh
scarlet@LTS:~$ ls -l ~/.ssh/authorized_keys
-rw----- 1 scarlet scarlet 97 Dec 14 20:07 /home/scarlet/.ssh/autho
rized_keys
scarlet@LTS:~$
```



Aseguramiento del entorno

```
scarlet@LTS:~$ sudo ufw status verbose
[sudo] password for scarlet:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
9090/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
9090/tcp (v6) ALLOW IN Anywhere (v6)

scarlet@LTS:~$ |
```

Configuración de UFW

El estado del firewall confirmó como activo, Status: active, con política de denegación predeterminada para el tráfico entrante, Default: deny (incoming). Las reglas mostraron que el acceso al puerto ssh, 22/tcp y al puerto del servicio Go, 9090/tcp, estaban configuradas para permitir tráfico entrante, ALLOW IN, tanto para IPv4 como para IPv6

Preparación del Binario

El usuario sin privilegios goservice tiene permisos para escribir la línea 'Servicio escuchado en el puerto 9090' en el archivo de log.

```
scarlet@LTS:~$ sudo cat /var/log/goservice.log
2025/12/14 20:26:38 Servicio escuchando en el puerto 9090
2025/12/14 20:26:42 Recibida señal de apagado, cerrando servicio...
2025/12/14 20:32:17 Servicio escuchando en el puerto 9090
2025/12/14 20:35:21 Nueva conexión desde [::1]:40964
2025/12/14 20:44:44 Nueva conexión desde 192.168.56.1:63623
2025/12/14 20:46:53 Nueva conexión desde 192.168.56.1:63710
2025/12/14 20:47:57 Nueva conexión desde 192.168.56.1:63715
2025/12/14 21:01:59 Recibida señal de apagado, cerrando servicio...
2025/12/14 21:02:01 Servicio escuchando en el puerto 9090
2025/12/14 21:15:47 Nueva conexión desde 192.168.56.1:62082
2025/12/14 21:17:56 Recibida señal de apagado, cerrando servicio...
2025/12/14 23:29:47 Servicio escuchando en el puerto 9090
2025/12/14 23:43:40 Recibida señal de apagado, cerrando servicio...
2025/12/14 23:45:09 Servicio escuchando en el puerto 9090
2025/12/14 23:54:52 Recibida señal de apagado, cerrando servicio...
2025/12/15 21:59:58 Servicio escuchando en el puerto 9090
```



Despliegue y Pruebas del Servicio

```
[Unit]
Description=Go TCP Service
After=network.target

[Service]
Type=simple
User=goservice
Group=goservice
WorkingDirectory=/opt/goservice
ExecStart=/opt/goservice/goservice
Restart=always
RestartSec=5
Environment=SERVICE_PORT=9090

[Install]
WantedBy=multi-user.target
```

Preparación del Binario

El comando `sudo systemctl status goservice` confirmó que el servicio estaba activo, se puede ver que dice active (running), habilitado al inicio del sistema, enabled, y ejecutándose con el PID asignado al usuario goservice.

Creación de systemd

Se configuró `User=goservice` para asegurar que el binario se ejecute con el usuario de mínimos privilegios y se estableció `Restart=always` para garantizar que el servicio se recupere automáticamente tras cualquier fallo

```
scarlet@LTS:~$ sudo systemctl daemon-reload
scarlet@LTS:~$ sudo systemctl enable goservice
scarlet@LTS:~$ sudo systemctl start goservice
scarlet@LTS:~$ sudo systemctl status goservice
● goservice.service - Go TCP Service
   Loaded: loaded (/etc/systemd/system/goservice.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-12-15 21:59:58 UTC; 21min ago
     Main PID: 790 (goservice)
        Tasks: 6 (limit: 2262)
       Memory: 3.0M (peak: 3.1M)
          CPU: 44ms
       CGroup: /system.slice/goservice.service
               └─790 /opt/goservice/goservice

Dec 15 21:59:58 LTS systemd[1]: Started goservice.service - Go TCP Service.
scarlet@LTS:~$
```



Despliegue y Pruebas del Servicio

```
scarlet@LTS:~$ nc localhost 9090
Servicio activo
```

Prueba Local

Se ejecutó una prueba conectividad local con nc localhost 9090, lo que resultó en la respuesta 'Servicio activo', probando que el daemon está escuchando correctamente en el puerto 9090.

Prueba Externa (HTTP)

El comando curl <http://192.168.56.102:9090/> se dirigió a la ip de la máquina virtual, resultando en una respuesta exitosa. El cual registró la nueva conexión entrante desde la ip del host, 192.168.56.1

```
C:\Users\Scarlet>curl http://192.168.56.102:9090/
```

```
scarlet@LTS:~$ sudo tail -f /var/log/goservice.log
2025/12/14 21:01:59 Recibida señal de apagado, cerrando servicio...
2025/12/14 21:02:01 Servicio escuchando en el puerto 9090
2025/12/14 21:15:47 Nueva conexión desde 192.168.56.1:62082
2025/12/14 21:17:56 Recibida señal de apagado, cerrando servicio...
2025/12/14 23:29:47 Servicio escuchando en el puerto 9090
2025/12/14 23:43:40 Recibida señal de apagado, cerrando servicio...
2025/12/14 23:45:09 Servicio escuchando en el puerto 9090
2025/12/14 23:54:52 Recibida señal de apagado, cerrando servicio...
2025/12/15 21:59:58 Servicio escuchando en el puerto 9090
2025/12/15 22:23:00 Nueva conexión desde [::1]:52414
```



Empaquetado y Portabilidad

```
GNU nano 7.2
Package: infoapp
Version: 1.0
Section: utils
Priority: optional
Architecture: amd64
Maintainer: Scarlet Abreu
Description: App que muestra ID, nombre y correo
```

Creación del Paquete

El paso final fue la creación del archivo de metadatos DEBIAN/control, el cual define atributos como el nombre del paquete, infoapp, la versión, 1.0, la arquitectura, amd64, y la descripción.

Construcción e Instalación

El paquete .deb se construyó utilizando el comando `dpkg-deb -build infoapp-deb` se procedió a la instalación con `sudo dpkg -i infoapp-deb.deb` que automáticamente colocó el binario infoapp en la ruta `usr/local/bin` definida en el paquete.

```
scarlet@LTS:~/infoapp$ dpkg-deb --build infoapp-deb
dpkg-deb: building package 'infoapp' in 'infoapp-deb.deb'.
scarlet@LTS:~/infoapp$ sudo dpkg -i infoapp-deb.deb
(Reading database ... 105212 files and directories currently installed.)
Preparing to unpack infoapp-deb.deb ...
Unpacking infoapp (1.0) over (1.0) ...
Setting up infoapp (1.0) ...
scarlet@LTS:~/infoapp$ |
```



Prueba de portabilidad

```
scarlet@scarlet:~$ git clone https://github.com/scarletabreu/goservice-systemd.git
Cloning into 'goservice-systemd'...
remote: Enumerating objects: 30, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 30 (delta 2), reused 24 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (30/30), 1.04 MiB | 2.06 MiB/s, done.
Resolving deltas: 100% (2/2), done.
scarlet@scarlet:~$ sudo dpkg -i goservice-systemd/deb-packaging/infoapp-deb.deb
Selecting previously unselected package infoapp.
(Reading database ... 86990 files and directories currently installed.)
Preparing to unpack .../deb-packaging/infoapp-deb.deb ...
Unpacking infoapp (1.0) ...
Setting up infoapp (1.0) ...
scarlet@scarlet:~$ infoapp
ID: 10153953
Nombre: Scarlet Abreu
Correo: SXAS0002@ce.pucmm.edu.do
scarlet@scarlet:~$ _
```

La prueba de despliegue se realizó clonando el repositorio del proyecto en una máquina virtual externa.

La instalación se ejecutó con éxito utilizando el administrador de paquetes `sudo dpkg -i infoapp-deb.deb`. "la ejecución del binario instalado mediante el comando `infoapp` mostró correctamente mis datos... confirmando la funcionalidad total y la portabilidad del paquete `.deb` generado.



Conclusión

- El proyecto demuestra el dominio en la gestión de servicios (systemd), asegurando la robustez y alta disponibilidad del daemon Go.
- Se implementó exitosamente un modelo de seguridad por capas, incluyendo autenticación estricta por SSH, el uso del Principio de Mínimo Privilegio y el control de acceso perimetral con UFW (Firewall).
- Se validó un flujo de distribución profesional mediante el empaquetado binario (dpkg), confirmando la portabilidad y la instalación limpia de la aplicación en entornos externos.
- **Logro Final:** Despliegue de un servicio Go funcional, seguro y automatizado en un ambiente simulado de producción Linux.

Enlace al repositorio: <https://github.com/scarletabreu/goservice-systemd>





Muchas Gracias Por Su Atención

